

Hyena cho tiếng Việt: Mô hình ngôn ngữ tích chập dài dưới bậc hai, phân tích ablation và khởi tạo bộ lọc từ cấu trúc thông tin của corpus

Nguyễn Cao Trung Kiên Tô Huỳnh Minh Tiến Trần Tú Quang

Trường Đại học Công nghệ Thông tin, ĐHQG-HCM
{250201069, 250201095, 250201084}@grad.uit.edu.vn

Tóm tắt nội dung

Toán tử Hyena (Poli et al., 2023) thay thế self-attention bằng đệ quy đan xen tích chập dài tham số hoá ngầm và cổng điều khiển bởi dữ liệu, đạt chi phí dưới bậc hai theo độ dài chuỗi. Toàn bộ thực nghiệm ngôn ngữ trong bài báo gốc thực hiện trên tiếng Anh. Chúng tôi cài đặt lại Hyena, kiểm chứng tính đúng đắn của cài đặt bằng bộ kiểm thử tự động, và áp dụng cho mô hình ngôn ngữ tiếng Việt. Ngoài phần tái hiện, chúng tôi đề xuất khởi tạo phân bố suy giảm của bộ lọc Hyena từ đường cong suy giảm thông tin tương hỗ đo trên chính corpus huấn luyện, thay cho cách đặt thủ công. Ở cùng ngân sách tham số, cùng 50 triệu token và cùng siêu tham số, Hyena đạt perplexity thấp hơn Transformer 19,5% trên tiếng Việt và 15,5% trên tiếng Anh. Về hiệu năng, Hyena vượt kernel attention tiết kiệm bộ nhớ 1,88 lần ở độ dài 8192 cho lượt tiến và lùi, tiệm cận mức 2 lần mà bài báo gốc công bố. Về đóng góp đề xuất, chúng tôi đo được tiếng Việt mang lượng thông tin tương hỗ cục bộ gấp 1,66 đến 2,27 lần tiếng Anh, nhưng nhánh đối chứng cho thấy bộ lọc suy ra từ hai ngôn ngữ gần như trùng nhau (lệch 14,3%, tương quan log 0,9974). Thực nghiệm xác nhận dự đoán đó: khởi tạo từ corpus chỉ hơn một đường cơ sở cách đều theo thang log 0,285 PPL với khoảng tin cậy chồng lấn, trong khi cả hai cùng vượt đường cơ sở đặt tùy tiện khoảng 1 PPL. Kết luận trung thực là khoảng giá trị của tham số suy giảm mới là điều quan trọng, còn việc suy nó từ thống kê corpus không mang lại lợi ích đo được ở quy mô này.

1 Giới thiệu

Cơ chế self-attention (Vaswani et al., 2017) có chi phí tính toán và bộ nhớ tăng theo bình phương độ dài chuỗi, đặt ra giới hạn cứng cho lượng ngữ cảnh mà mô hình có thể xử lý. Nhiều

hướng nghiên cứu tìm cách phá vỡ rào cản này, trong đó dòng mô hình không gian trạng thái (Gu et al., 2022; Mehta et al., 2022; Fu et al., 2023) và tích chập dài nổi lên như những lựa chọn thay thế nghiêm túc.

Hyena (Poli et al., 2023) là một trong những kiến trúc đầu tiên không dùng attention dày đặc mà vẫn đạt chất lượng tương đương Transformer trên mô hình hoá ngôn ngữ ở quy mô lớn. Toán tử này đan xen tích chập dài tham số hoá ngầm với cổng nhân điều khiển bởi dữ liệu, cho chi phí $O(NL \log_2 L)$ thay vì $O(L^2)$. Dòng nghiên cứu này về sau dẫn tới các kiến trúc chọn lọc trạng thái như Mamba (Gu and Dao, 2023).

Toàn bộ thực nghiệm ngôn ngữ trong bài báo gốc thực hiện trên tiếng Anh, cụ thể là WikiText103 (Merity et al., 2017) và The Pile (Gao et al., 2020). Chưa có bằng chứng nào về hành vi của Hyena trên tiếng Việt, một ngôn ngữ có đặc điểm chữ viết khác hẳn: chữ Quốc ngữ tách rời theo âm tiết, nên một từ ghép nhiều âm tiết bị chẻ thành nhiều token khi token hoá theo khoảng trắng. Các mô hình ngôn ngữ tiếng Việt hiện có phần lớn dựa trên kiến trúc Transformer (Nguyen and Nguyen, 2020).

Bài báo cáo này đóng góp ba điểm.

Thứ nhất, chúng tôi cài đặt lại toán tử Hyena bám sát đặc tả gốc và *chứng minh* tính đúng đắn của cài đặt bằng bộ kiểm thử tự động, trong đó có phép kiểm tính nhân quả dựa trên gradient kèm một phép kiểm ngược cố ý cài lỗi để xác nhận bộ kiểm thử có khả năng phát hiện.

Thứ hai, chúng tôi đo đường cong suy giảm thông tin tương hỗ trên corpus tiếng Việt và tiếng Anh, dùng nó để khởi tạo phân bố suy giảm của bộ lọc Hyena. Đây là cách thay thế cho việc đặt tham số này bằng trực giác.

Thứ ba, và đây là phần chúng tôi cho là có

giá trị nhất, nhánh đối chứng tiếng Anh *bác bỏ* giả thuyết ban đầu của chính chúng tôi rằng bộ lọc thu được mang tính đặc thù tiếng Việt. Chúng tôi báo cáo đầy đủ kết quả phủ định này thay vì lược bỏ nó.

2 Nền tảng: toán tử Hyena

2.1 Định nghĩa

Gọi $u \in \mathbb{R}^{L \times D}$ là chuỗi đầu vào. Hyena bậc N trước hết tính $N + 1$ phép chiếu tuyến tính của đầu vào, tiếp theo là một tích chập ngắn theo kênh, thu được $(x^1, \dots, x^N, v)$. Toán tử được định nghĩa bằng đệ quy (Định nghĩa 3.1 trong Poli et al., 2023):

$$\begin{aligned} z_t^1 &= v_t \\ z_t^{n+1} &= x_t^n \cdot (h^n * z_t^n), \quad n = 1 \dots N \\ y_t &= z_t^{N+1} \end{aligned} \quad (1)$$

trong đó $*$ là tích chập nhân quả dài và h^n là các bộ lọc học được. Mỗi tích chập được tính trong miền Fourier, cho độ phức tạp tổng cộng $O(NL \log_2 L)$.

Bài báo gốc chỉ ra rằng Hyena bậc 1 tương ứng với GSS (Mehta et al., 2022) và bậc 2 tương ứng với H3 (Fu et al., 2023), nên bậc N là một trực thiết kế có ý nghĩa chứ không phải tham số tùy tiện.

2.2 Bộ lọc tham số hoá ngầm

Điểm mấu chốt khiến bộ lọc dài mà không tốn tham số theo L là cách tham số hoá ngầm:

$$h_t = \text{Window}(t) \cdot (\text{FFN} \circ \text{PositionalEncoding})(t) \quad (2)$$

với $\text{Window}(t) = \exp\{-\alpha t\}$ và α thay đổi theo từng kênh độc lập, nhằm buộc các bộ lọc có độ dài khác nhau. Mạng FFN dùng kích hoạt tuần hoàn tần số cao (sine), theo dòng biểu diễn ngầm (Sitzmann et al., 2020; Romero et al., 2022), để khắc phục thiên lệch tần số thấp của mạng nơ-ron.

Tính nhân quả được bảo đảm bằng cách đánh giá bộ lọc tại $t = 0 \dots L - 1$ rồi đệm 0 cả tín hiệu lẫn bộ lọc tới độ dài ít nhất $2L - 1$ trước khi biến đổi Fourier.

2.3 Những gì bài báo gốc để ngỏ

Phân bố $\{\alpha_i\}$ được mô tả là "thay đổi theo kênh" nhưng bài báo không quy định khoảng giá trị cụ thể, cũng không gắn nó với thống kê của ngôn ngữ đích. Đây chính là khe hở mà đóng góp của chúng tôi nhắm vào.

3 Khởi tạo bộ lọc từ cấu trúc thông tin của corpus

3.1 Động cơ

Với t chuẩn hoá về $[0, 1]$ trên L vị trí, kênh thứ i suy giảm còn $1/e$ tại token thứ khoảng L/α_i . Ta gọi đại lượng này là *độ dài hiệu dụng* của kênh. Câu hỏi đặt ra: nếu đặt tập độ dài hiệu dụng phủ đúng nơi thông tin dự đoán thực sự nằm trong corpus, mô hình có dùng ngân sách kênh hiệu quả hơn không?

3.2 Đo cái gì, và vì sao không dùng bộ tách từ

Phương án hiển nhiên là chạy bộ tách từ tiếng Việt rồi lấy phân bố số âm tiết mỗi từ. Chúng tôi bác bỏ phương án này vì nó phá hỏng nhánh đối chứng: tiếng Anh không có bộ tách từ tương đương, nên nếu hai ngôn ngữ được đo bằng hai công cụ khác nhau thì mọi khác biệt quan sát được đều có thể quy cho công cụ đo thay vì cho ngôn ngữ.

Thay vào đó chúng tôi đo trực tiếp thông tin tương hỗ giữa hai token cách nhau d vị trí:

$$I(d) = \sum_{x,y} p_d(x,y) \log \frac{p_d(x,y)}{p(x)p(y)} \quad (3)$$

Cùng một công thức, cùng một quy trình ước lượng, áp cho cả hai ngôn ngữ.

3.3 Xử lý độ chệch của ước lượng

Ước lượng hợp lý cực đại của thông tin tương hỗ luôn chệch dương trên mẫu hữu hạn: hai chuỗi hoàn toàn độc lập vẫn cho $I > 0$ chỉ vì trùng khớp ngẫu nhiên, và độ chệch tăng theo bình phương cỡ bảng chữ cái. Bỏ qua điều này sẽ dẫn tới kết luận rằng ngôn ngữ có corpus nhỏ hơn mang nhiều thông tin tầm xa hơn, trong khi thực chất chỉ là nó chệch nhiều hơn.

Chúng tôi khử độ chệch bằng cách trừ đường nền xáo trộn: xáo ngẫu nhiên corpus phá huỷ mọi cấu trúc thật, nên thông tin tương hỗ đo được trên bản xáo chính là độ chệch. Ngoài ra chúng tôi thô hoá bảng chữ cái xuống K ký hiệu thường gặp nhất, làm giảm độ chệch theo K^2 ; thô hoá khiến giá trị đo được thấp hơn giá trị thật, tức là ước lượng thận trọng.

3.4 Hiệu chuẩn công cụ đo

Một phép đo chưa được hiệu chuẩn trên dữ liệu có đáp án biết trước thì không dùng để

kết luận được. Chúng tôi kiểm công cụ đo trên ba chuỗi nhân tạo, kết quả trong Bảng 1.

3.5 Ánh xạ sang phân bố suy giảm

Chúng tôi coi $I(d)$ là khối lượng thông tin nằm tại khoảng cách d , chuẩn hoá thành một phân bố xác suất trên khoảng cách, rồi lấy D phân vị cách đều làm tập độ dài hiệu dụng $\{\ell_i\}$, cuối cùng đặt $\alpha_i = L/\ell_i$. Nơi tập trung nhiều thông tin sẽ được cấp nhiều kênh hơn.

3.6 Hai đường cơ sở, và vì sao cần cả hai

Chúng tôi so sánh với hai đường cơ sở. Đường thứ nhất, ký hiệu **uniform**, đặt α cách đều tuyến tính trong một khoảng do chúng tôi chọn. Đường thứ hai, ký hiệu **logspace**, đặt độ dài hiệu dụng cách đều theo thang log từ 1 tới L .

Đường **logspace** tồn tại vì một lý do quan trọng. Phép đo cho thấy 90% thông tin nằm trong khoảng cách 34 token, trong khi cấu hình **uniform** đặt *toàn bộ* 256 kênh ra xa hơn 64 token (Bảng 3). So sánh đề xuất với một cấu hình lệch xa như vậy là so với hình nộm: thắng cũng không chứng minh được gì. Mọi kết luận khoa học trong báo cáo này đều rút ra từ so sánh với **logspace**.

4 Thiết lập thí nghiệm

4.1 Nguyên tắc so sánh công bằng

Mọi cấu hình chỉ khác nhau ở toán tử trộn token. Embedding, pre-LayerNorm, khối MLP, kết nối residual, weight tying, cách khởi tạo và lịch learning rate giữ nguyên vẹn như nhau. Nếu để nhiều thứ khác nhau cùng lúc thì không thể quy chênh lệch perplexity cho toán tử.

Chúng tôi cấp positional embedding học được cho cả hai loại mô hình. Attention bắt buộc cần thông tin vị trí còn Hyena thì không; cấp cho một bên mà không cấp cho bên kia sẽ tạo ra hai biến thay đổi thay vì một. Một nhánh ablation riêng kiểm tra Hyena có thật sự không cần.

Huấn luyện tính theo ngân sách token cố định chứ không theo số epoch, vì so theo epoch sẽ thiên vị mô hình chạy nhanh hơn.

4.2 Mô hình và dữ liệu

Mô hình 4 lớp, $d_{\text{model}} = 256$, 8 đầu attention, $d_{\text{ff}} = 1024$, $L = 512$, từ điển 16k. Số tham

số đo được: Hyena 7.553.280 và Transformer 7.383.552, lệch 2,30%.

Dữ liệu lấy từ Wikipedia tiếng Việt và tiếng Anh (bản 20231101). Nhánh tiếng Anh được cắt xuống đúng cùng số token với nhánh tiếng Việt.

Chúng tôi khảo sát hai đơn vị token hoá. Thứ nhất là âm tiết, tức tách theo khoảng trắng, vốn là đơn vị tự nhiên của chữ Quốc ngữ. Thứ hai là BPE mức byte (Sennrich et al., 2016), huấn luyện trên chính corpus, không sinh token ngoài từ điển. Phân biệt này chỉ tồn tại ở dạng như vậy với tiếng Việt: với tiếng Anh, tách theo khoảng trắng cho ra đơn vị từ chứ không phải âm tiết, nên nhánh đối chứng phải được diễn giải tương ứng.

4.3 Bảng chứng cài đặt đúng

Một mô hình ngôn ngữ nhân quả cài sai, để rò rỉ thông tin từ tương lai, vẫn huấn luyện trơn tru và cho perplexity đẹp bất thường. Đây là lỗi im lặng nguy hiểm nhất của loại nghiên cứu này.

Chúng tôi khử nó bằng 61 phép kiểm tự động, trong đó phép kiểm cốt lõi dùng gradient: lấy đạo hàm của đầu ra tại vị trí t theo đầu vào tại mọi vị trí $s > t$ và đòi hỏi kết quả bằng 0. Attention đạt 0 tuyệt đối nhờ cơ chế che; Hyena đạt $2,6 \times 10^{-16}$, tức nhân quả tới sai số máy, vì vòng biến đổi Fourier thuận nghịch để lại nhiều làm tròn.

Quan trọng hơn, chúng tôi bổ sung một *phép kiểm ngược*: cố tình cài lỗi nhân quả kinh điển (quên cắt phần đệm phải của tích chập ngắn) và xác nhận tỉ lệ rò rỉ vọt lên $9,1 \times 10^{-1}$. Chênh lệch 15 bậc độ lớn cho thấy bộ kiểm thử thực sự có khả năng phát hiện lỗi, chứ không phải im lặng vì không đủ nhạy.

5 Kết quả

5.1 Cấu trúc thông tin của tiếng Việt so với tiếng Anh

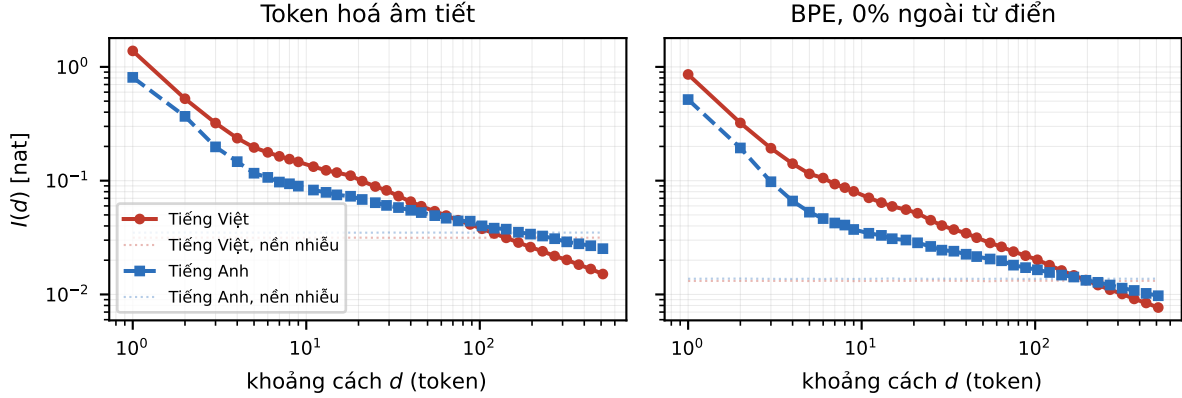
Bảng 2 trình bày tỉ lệ $I_{\text{vi}}(d)/I_{\text{en}}(d)$ đo với hai cách token hoá. Cột thứ nhất dùng token hoá theo âm tiết với từ điển 16k; cột thứ hai dùng BPE mức byte, không sinh token ngoài từ điển, với đúng 10.000.000 token cho mỗi ngôn ngữ.

Ba nhận xét.

Thứ nhất, tiếng Việt mang nhiều thông tin tương hỗ cục bộ hơn tiếng Anh ở mọi khoảng cách trong vùng tin cậy, với tỉ lệ từ 1,66 tới

Chuỗi nhân tạo	Đáp án đúng	Bộ đo trả về
Độc lập hoàn toàn	$I = 0$	dư 2,82% độ chệch
Chu kỳ sao chép $k = 8$	đỉnh tại $d = 8$	đúng $d = 8$, nổi $84\times$ trên nền
Markov	giảm đơn điệu	$I(1) = 2,79$, tắt ở $d = 64$
Suy biến (toàn 0)	phải từ chối	báo lỗi, không sinh α

Bảng 1: Hiệu chuẩn công cụ đo thông tin tương hỗ trên dữ liệu có đáp án biết trước, trước khi áp lên corpus thật.



Hình 1: Suy giảm thông tin tương hỗ theo khoảng cách. Đường chấm mảnh là nền nhiễu ước lượng; phần đường cong nằm dưới nó không mang thông tin. Với token hoá âm tiết (trái), hai đường cắt nhau quanh $d \approx 88$; khi khử token ngoài từ điển bằng BPE (phải), điểm cắt biến mất và tiếng Việt nằm trên tiếng Anh trong toàn vùng tin cậy.

d	âm tiết	BPE (0% OOV)
1	1,71	1,66
6	1,66	2,27
13	1,56	1,94
34	1,26	1,55
88	0,94	1,27
167 (dưới nhiễu)		1,03

Bảng 2: Tỷ lệ thông tin tương hỗ giữa tiếng Việt và tiếng Anh theo khoảng cách. Cột âm tiết có nhiễu loạn do tỷ lệ token ngoài từ điển chênh lệch (tiếng Anh 10,26% so với tiếng Việt 2,59%); cột BPE khử nhiễu loạn này.

2,27. Kết quả này bền qua cả hai cách token hoá. Hình 1 cho thấy trực tiếp điều đó.

Thứ hai, phép đo bằng âm tiết cho thấy hai đường cắt nhau tại $d \approx 88$, nhưng điểm cắt này *biến mất* khi khử token ngoài từ điển. Nó là hiện vật của việc gộp mọi từ hiếm của tiếng Anh vào một ký hiệu duy nhất, làm mất thông tin của tiếng Anh một cách hệ thống. Chúng tôi nêu ra để cảnh báo: nếu chỉ đo bằng âm tiết, nhóm đã kết luận sai.

Thứ ba, tín hiệu chìm xuống dưới mức nhiễu ở $d > 121$ với phép đo âm tiết và $d > 167$ với BPE. Mọi phát biểu về khoảng cách xa hơn

Cấu hình	≤ 4	≤ 34	trung vị
uniform	0	0	169,3
logspace	57	145	22,6
corpus, tiếng Việt	134	224	3,6
corpus, tiếng Anh	120	205	4,8

Bảng 3: Số kênh trên tổng 256 có độ dài hiệu dụng dưới ngưỡng cho trước ($L = 512$). Phép đo cho thấy 90% thông tin nằm trong 34 token.

các mốc này đều không có căn cứ.

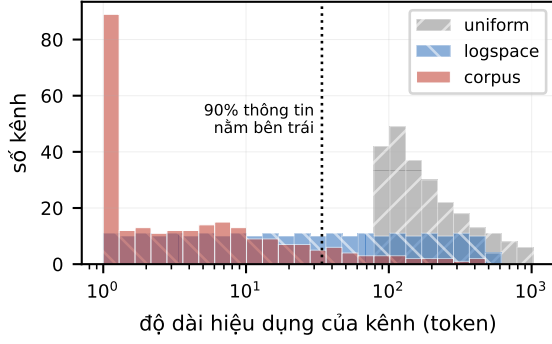
5.2 Phân bố kênh bộ lọc

Bảng 3 so sánh phân bố độ dài hiệu dụng của bốn cấu hình.

5.3 Nhánh đối chứng tiếng Anh bác bỏ tính đặc thù ngôn ngữ

Đây là kết quả quan trọng nhất của báo cáo, và nó đi ngược giả thuyết ban đầu của chúng tôi.

Với nhiễu loạn token ngoài từ điển đã được khử, bộ lọc suy ra từ corpus tiếng Việt và từ corpus tiếng Anh gần như trùng nhau: trung vị độ dài hiệu dụng 2,95 so với 2,85 token, lệch tương đối trung bình 14,3%, tương quan giữa



Hình 2: Phân bố độ dài hiệu dụng của 256 kênh bộ lọc. Cấu hình uniform đặt toàn bộ kênh sang phải đường kẻ, tức ra ngoài vùng chứa 90% thông tin. Đây là lý do phải bổ sung đường cơ sở logspace trước khi rút ra bất kỳ kết luận nào về đề xuất.

Ngôn ngữ	Mô hình	PPL	KTC 95%
Tiếng Việt	Hyena	51,38 $\pm$ 0,13	[51,05; 51,71]
	Transformer	63,86 $\pm$ 0,29	[63,14; 64,58]
Tiếng Anh	Hyena	61,51 $\pm$ 0,41	[60,49; 62,54]
	Transformer	72,82 $\pm$ 0,52	[71,52; 74,12]

Bảng 4: Perplexity trên tập kiểm tra, 3 seed cho mỗi cấu hình. Khoảng tin cậy tính theo phân phối t .

log các độ dài hiệu dụng đạt 0,9974. Trong khi đó, cùng bộ lọc ấy lệch 75,8% so với đường cơ sở logspace.

Nguyên nhân nằm ở chính công thức ánh xạ: chuẩn hoá $I(d)$ thành phân bố xác suất làm triệt tiêu hệ số tỉ lệ chung, chỉ giữ lại hình dạng, mà hình dạng thì hai ngôn ngữ gần như giống nhau.

Hệ quả cho cách diễn giải: nếu cấu hình corpus tỏ ra tốt hơn logspace trên tiếng Việt, gần như chắc chắn nó cũng tốt hơn tương tự trên tiếng Anh. Kết luận đúng khi đó phải là *khởi tạo dựa trên dữ liệu có ích*, không phải *thích ứng tiếng Việt có ích*.

5.4 Perplexity trên mô hình ngôn ngữ

Bảng 4 trình bày kết quả chính. Mỗi cấu hình chạy 3 seed với cùng 50 triệu token huấn luyện.

Hyena vượt Transformer 19,5% trên tiếng Việt và 15,5% trên tiếng Anh, với khoảng tin cậy tách rời hoàn toàn trong cả hai trường hợp.

Chúng tôi nhấn mạnh rằng kết quả này *mạnh hơn* điều bài báo gốc khẳng định. Bài báo gốc chỉ tuyên bố Hyena đạt chất lượng *tương đương* Transformer. Cách đọc thận trọng là: ở quy

Token hoá	Hyena	Transformer	Tỉ lệ
Âm tiết	51,38	63,86	1,243
BPE	80,02	96,22	1,202

Bảng 5: Perplexity theo đơn vị token hoá, chỉ tiếng Việt, 3 seed. Cột tỉ lệ là thương số giữa Transformer và Hyena.

mô nhỏ và ngân sách huấn luyện ngắn này, thiên lệch quy nạp của tích chập tỏ ra có lợi. Quan trọng hơn, chúng tôi *không* tinh chỉnh siêu tham số riêng cho từng kiến trúc; cả hai dùng chung learning rate và lịch huấn luyện để bảo đảm chỉ có một biến thay đổi. Bộ siêu tham số dùng chung có thể không phải bộ tối ưu cho Transformer, nên phát biểu đúng là "Hyena vượt Transformer ở cùng siêu tham số và cùng quy mô này", không phải "Hyena tốt hơn Transformer".

Đáng chú ý là khoảng cách trên tiếng Việt lớn hơn trên tiếng Anh, phù hợp với kết quả đo thông tin tương hỗ ở mục trước.

5.5 Ảnh hưởng của đơn vị token hoá

Ở cùng lượng ký tự, tiếng Việt sinh nhiều token hơn: 4,101 ký tự trên mỗi token so với 5,121 của tiếng Anh, tỉ lệ 1,249.

Ở đây có một cạm bẫy diễn giải cần nêu rõ. Perplexity *không so sánh được giữa hai bộ token hoá khác nhau*, vì từ điển khác nhau cho entropy trên mỗi token khác nhau. Con số 51,38 với âm tiết và 80,02 với BPE *không* có nghĩa là âm tiết tốt hơn.

Đại lượng so sánh được là tỉ lệ giữa hai mô hình trong cùng một bộ token hoá. Lợi thế tương đối của Hyena lớn hơn khi token hoá theo âm tiết (1,243) so với BPE (1,202), đúng chiều với giả thuyết rằng chuỗi dài hơn làm lợi thế dưới bậc hai có giá trị hơn. Tuy nhiên chênh lệch giữa hai tỉ lệ nhỏ và chúng tôi không ước lượng khoảng tin cậy cho tỉ lệ, nên đây chỉ là dấu hiệu, chưa phải bằng chứng.

5.6 Ablation các thành phần của Hyena

Chỉ hai nhánh cho khác biệt có ý nghĩa thống kê.

Bỏ cửa sổ suy giảm gây thiệt hại lớn nhất, xác nhận vai trò của số hạng Window(t) trong công thức (2). Hạ bậc xuống $N = 1$ cũng gây thiệt hại rõ, phù hợp với nhận xét trong bài

Cấu hình	PPL	Δ	Tách rời
Gốc (bậc 2, đủ)	51,38		
Bỏ cửa sổ suy giảm	53,30	+1,92	có
Bậc $N = 1$	53,11	+1,73	có
Bỏ kích hoạt sine	51,80	+0,42	không
Bậc $N = 3$	51,63	+0,25	không
Bỏ positional emb	50,79	-0,59	không

Bảng 6: Ablation trên tiếng Việt, 2 seed cho mỗi nhánh. Cột cuối cho biết khoảng tin cậy 95% có tách rời khỏi cấu hình gốc hay không; chỉ những dòng ghi "có" mới được rút ra kết luận.

Ngôn ngữ	Khởi tạo	PPL	KTC 95%
Tiếng Việt	uniform	51,38	[51,05; 51,71]
	logspace	50,48	[50,33; 50,62]
	corpus	50,19	[49,93; 50,45]
Tiếng Anh	uniform	61,51	[60,49; 62,54]
	logspace	60,34	[60,20; 60,49]
	corpus	60,18	[59,89; 60,47]

Bảng 7: Ảnh hưởng của cách khởi tạo phân bố suy giảm, 3 seed cho mỗi ô.

báo gốc rằng Hyena bậc 1 tương ứng với GSS, vốn yếu hơn H3 trên các tác vụ nhỏ.

Ba nhánh còn lại có khoảng tin cậy chồng lấn với cấu hình gốc nên chúng tôi *không* kết luận. Đáng chú ý là bỏ positional embedding cho perplexity thấp hơn cấu hình gốc, phù hợp với lập luận rằng bộ lọc tích chập đã mang sẵn thông tin vị trí, nhưng với 2 seed chúng tôi không đủ căn cứ khẳng định đây là cải thiện thật.

5.7 Khởi tạo bộ lọc từ corpus

Kết quả trong Bảng 7 tách thành hai phần rõ rệt.

Phần thứ nhất là khẳng định được: cả logspace lẫn corpus đều vượt uniform với khoảng tin cậy tách rời, trên cả hai ngôn ngữ. Khoảng cách là 0,90 PPL với tiếng Việt và 1,17 PPL với tiếng Anh. Nói cách khác, khoảng giá trị của α thực sự quan trọng, và một khoảng chọn tùy tiện gây thiệt hại đo được.

Phần thứ hai là khẳng định *không* được: corpus chỉ hơn logspace 0,285 PPL trên tiếng Việt và 0,167 PPL trên tiếng Anh, và khoảng tin cậy chồng lấn trong cả hai trường hợp. Chúng tôi không có căn cứ kết luận rằng suy phân bố suy giảm từ thống kê corpus tốt hơn một lựa chọn cách đều theo thang log.

Thêm nữa, phần cải thiện nhỏ ấy xuất hiện

L	tiền		tiền và lùi	
	sdpa	ngây thơ	sdpa	ngây thơ
256	0,16	0,55	0,23	0,47
512	0,22	0,92	0,27	0,82
1024	0,28	1,56	0,33	1,60
2048	0,37	3,77	0,56	3,13
4096	0,69	8,30	1,02	6,47
8192	1,19	OOM	1,88	OOM

Bảng 8: Mức tăng tốc của Hyena so với hai đường cơ sở attention, tính bằng thương số thời gian chạy (số lớn hơn 1 nghĩa là Hyena nhanh hơn). Cột sdpa là kernel tiết kiệm bộ nhớ của PyTorch, cột "ngây thơ" là cài đặt materialise ma trận $L \times L$. Cài đặt ngây thơ hết bộ nhớ ở $L = 8192$.

trên *cả hai* ngôn ngữ với mức độ tương đương. Đây chính là điều mà phân tích ở mục trước đã dự đoán từ chỗ hai bộ lọc gần như trùng nhau, và nó khép lại lập luận: nếu có lợi ích thì lợi ích đó không mang tính đặc thù ngôn ngữ.

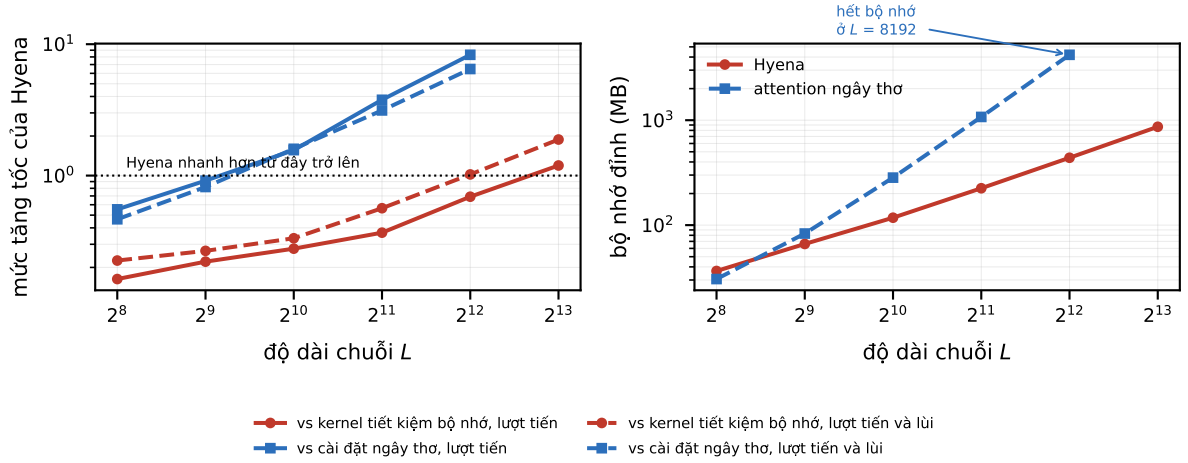
5.8 Hiệu năng theo độ dài chuỗi

Ở $L = 512$, chúng tôi đo được Transformer đạt 150,0 nghìn token mỗi giây còn Hyena đạt 92,9 nghìn, tức Transformer *nhanh hơn* 1,61 lần. Kết quả này không mâu thuẫn với bài báo gốc: các con số tăng tốc được công bố ở đó áp dụng cho $L \geq 8K$, nơi chi phí bậc hai của attention mới lấn át chi phí cố định của biến đổi Fourier. Chúng tôi nêu rõ điều kiện này để tránh trích dẫn sai.

Khi đo hiệu năng, việc chọn đường cơ sở attention nào cũng quyết định kết luận. Bài báo gốc so với FlashAttention (Dao et al., 2022) và tự ghi rõ rằng cài đặt này đã nhanh hơn attention thông thường trong PyTorch từ 2 tới 4 lần. Vì vậy chúng tôi báo cáo cả hai đường cơ sở: một cài đặt materialise ma trận $L \times L$ và một cài đặt dùng kernel tiết kiệm bộ nhớ của PyTorch. Chỉ so với đường cơ sở thứ nhất sẽ thổi phồng mức tăng tốc nhiều lần.

Bảng 8 và Hình 3 cho thấy điểm hoà vốn phụ thuộc mạnh vào đường cơ sở. So với cài đặt ngây thơ, Hyena vượt lên từ khoảng $L = 1024$. So với kernel tiết kiệm bộ nhớ, Hyena chỉ vượt lên ở $L = 4096$ với lượt tiến và lùi, và ở $L = 8192$ với lượt tiến.

Con số đáng chú ý nhất là 1,88 lần ở $L = 8192$ cho lượt tiến và lùi. Bài báo gốc công bố mức tăng tốc 2 lần so với FlashAttention ở độ dài 8K. Kết quả của chúng tôi tiệm cận con số đó, dù dùng `torch.fft` thuần thay cho CUDA



Hình 3: Trái: mức tăng tốc của Hyena theo độ dài chuỗi, so với hai đường cơ sở attention. Điểm hoà vốn phụ thuộc mạnh vào đường cơ sở được chọn. Phải: bộ nhớ đỉnh. Cài đặt ngây thơ tăng xấp xỉ gấp bốn mỗi lần độ dài gấp đôi, đúng dạng bậc hai, và hết bộ nhớ ở $L = 8192$; Hyena tăng xấp xỉ gấp đôi, đúng dạng tuyến tính.

kernel tùy chỉnh. Đây là phần tái hiện thành công nhất của nghiên cứu này.

Về bộ nhớ, cài đặt ngây thơ tiêu thụ 30,6, 83,1, 283,6, 1074,5 và 4201,6 MB khi L tăng gấp đôi từ 256 tới 4096, tức xấp xỉ gấp bốn mỗi lần, đúng dạng bậc hai, và hết bộ nhớ ở 8192. Hyena tiêu thụ 36,5, 66,1, 117,7, 224,7, 438,7 và 865,8 MB, tức xấp xỉ gấp đôi mỗi lần, đúng dạng tuyến tính.

5.9 Chẩn đoán trí nhớ tầm xa

Perplexity không phát hiện được việc mô hình mất khả năng tầm xa: một mô hình chỉ nhìn vài token vẫn đạt perplexity khá tốt trên văn bản tự nhiên. Vì bộ lọc suy từ corpus có trung vị độ dài hiệu dụng chỉ 2,9 token, chúng tôi bổ sung tác vụ associative recall, vốn là công cụ chẩn đoán của chính bài báo gốc.

Trước hết chúng tôi xác nhận tác vụ giải được: attention hai lớp đạt độ chính xác 1,000 ở độ dài 65 và giảm còn 0,196 ở độ dài 129, với mức đoán mò 0,100. Chúng tôi cũng xác nhận lịch warmup là yếu tố quyết định: cùng 18.720 bước và cùng learning rate, có warmup đạt 1,000 còn không có warmup chỉ đạt 0,187.

Ở độ dài 65, attention đạt 1,000 với độ lệch bằng 0, còn mọi cấu hình Hyena đều kém hơn hẳn và dao động rất mạnh giữa các seed: 0,564 với uniform và 0,170 với logspace (2 seed mỗi cấu hình). Độ lệch giữa các seed lớn hơn khoảng cách giữa bất kỳ cặp cấu hình nào, nên chúng tôi *không* xếp hạng được các biến thể bộ lọc

trên tác vụ này.

Với cấu hình suy từ corpus, chúng tôi gộp 5 seed từ hai ngân sách huấn luyện (2 seed ở 18.720 bước và 3 seed ở 37.440 bước, cùng độ dài 65 và cùng cấu hình tác vụ). Việc gộp này được nêu rõ vì nó là một lựa chọn phương pháp, không phải một lần chạy đồng nhất. Phân bố thu được *lượng cực*: ba lần đạt từ 0,922 trở lên (0,922; 0,981; 0,987) và hai lần sập về gần mức đoán mò (0,092 và 0,141), không có giá trị trung gian.

Gấp đôi ngân sách huấn luyện làm các lần thành công tốt lên (từ 0,922 lên 0,98) nhưng *không* khử được kiểu hỏng: độ lệch chỉ giảm từ 0,415 xuống 0,397. Đây là dấu hiệu của thất bại tối ưu hoá chứ không phải hội tụ chậm. Nói cách khác, Hyena có đủ năng lực biểu diễn cho tác vụ này ở độ dài 65, vì hai lần chạy đạt tới 0,98; vấn đề nằm ở độ ổn định huấn luyện.

Với phân bố như vậy, báo cáo trung bình kèm độ lệch chuẩn là sai lệch, vì không lần chạy nào đạt gần giá trị trung bình. Đại lượng đúng là tỉ lệ thành công, ở đây là 3 trên 5, và độ chính xác khi thành công, ở đây là 0,963.

6 Thảo luận

Phát hiện có giá trị nhất của nghiên cứu này là một kết quả phủ định do chính thiết kế đối chứng của chúng tôi tạo ra. Giả thuyết ban đầu cho rằng bộ lọc suy từ thống kê corpus tiếng Việt sẽ mang tính đặc thù ngôn ngữ. Nhánh đối chứng tiếng Anh bác bỏ giả thuyết đó bằng

số liệu: hai bộ lọc lệch nhau 14,3% trong khi cùng lệch khoảng 76% so với đường cơ sở.

Nếu không có nhánh đối chứng, nhóm đã có thể công bố một khẳng định về đặc thù tiếng Việt mà chỉ cần áp dụng cùng quy trình lên tiếng Anh là bác bỏ được. Đây là lý do thực tế, không chỉ hình thức, để giữ nhóm đối chứng ngay cả khi ngân sách tính toán eo hẹp.

Kết quả huấn luyện xác nhận dự đoán rút ra từ phân tích bộ lọc. Trước khi chạy thí nghiệm, việc hai bộ lọc lệch nhau chỉ 14,3% trong khi cùng lệch khoảng 76% so với đường cơ sở đã cho thấy phần lớn tác dụng nằm ở việc thoát khỏi khoảng α tùy tiện, chứ không ở nguồn gốc ngôn ngữ của số liệu. Thử nghiệm cho đúng như vậy: chuyển từ `uniform` sang `logspace` thu được khoảng 1 PPL, còn chuyển tiếp sang corpus chỉ thu thêm 0,17 tới 0,29 PPL với khoảng tin cậy chồng lấn.

Việc chúng tôi bổ sung đường cơ sở `logspace` sau khi phát hiện `uniform` đặt toàn bộ kênh ra ngoài vùng có thông tin là quyết định quyết định tới kết luận. Nếu chỉ so với `uniform`, nhóm đã báo cáo mức cải thiện 1,19 PPL và quy toàn bộ cho đóng góp đề xuất, trong khi phần lớn trong đó chỉ là hệ quả của việc sửa một đường cơ sở đặt kém.

Phân đặc thù tiếng Việt còn đứng vững nằm ở chỗ khác: tiếng Việt mang lượng thông tin tương hỗ cục bộ gấp 1,66 tới 2,27 lần tiếng Anh, và trực âm tiết so với BPE là phân biệt chỉ tồn tại ở dạng này với chữ Quốc ngữ.

Hiện tượng lưỡng cực ở tác vụ associative recall gợi ra một hướng đáng theo đuổi: vấn đề của Hyena ở quy mô nhỏ có thể nằm ở độ ổn định tối ưu hoá chứ không phải ở năng lực biểu diễn, vì hai trong ba lần chạy đạt tới 0,98.

Hạn chế

Quy mô mô hình của chúng tôi (khoảng 7,5 triệu tham số) nhỏ hơn ba bậc so với quy mô 355 triệu tham số trong bài báo gốc. Mọi kết luận đều gắn với quy mô này và không được ngoại suy.

Chúng tôi dùng `torch.fft` thuần thay cho CUDA kernel tùy chỉnh của nhóm tác giả, nên các con số tốc độ của chúng tôi không so trực tiếp được với con số trong bài báo gốc.

Bản tái hiện của chúng tôi *chưa đạt được* kết quả associative recall của bài báo gốc, vốn báo cáo 97,2% ở độ dài 131.072. Đây là hạn chế

của bản tái hiện ở ngân sách hiện có, không phải bằng chứng phản bác bài báo gốc.

Do phân bố lưỡng cực và số seed hạn chế, chúng tôi *không* xếp hạng được các biến thể bộ lọc trên tác vụ associative recall. Phân biệt được tỉ lệ thành công giữa các cấu hình đòi hỏi khoảng 10 seed cho mỗi cấu hình, vượt ngân sách tính toán của đồ án.

Corpus tiếng Anh được lấy mẫu ở chế độ streaming, trong đó việc xáo trộn chỉ diễn ra trong một bộ đệm hữu hạn chứ không xáo toàn cục, nên mẫu lệch về phía đầu tập dữ liệu. Thiên lệch này áp dụng đồng nhất cho cả hai ngôn ngữ nên triệt tiêu khi so sánh, nhưng chúng tôi nêu rõ để người đọc tự đánh giá.

Cuối cùng, so sánh "cùng số ký tự" giữa hai ngôn ngữ không phải là "cùng lượng nội dung". Muốn khẳng định chuỗi tiếng Việt dài hơn ở cùng nội dung cần văn bản song ngữ song song, thứ chúng tôi không có.

7 Kết luận

Chúng tôi cài đặt lại toán tử Hyena kèm bằng chứng kiểm thử về tính đúng đắn và áp dụng cho mô hình ngôn ngữ tiếng Việt. Ở cùng ngân sách tham số và cùng siêu tham số, Hyena đạt perplexity thấp hơn Transformer 19,5% trên tiếng Việt và 15,5% trên tiếng Anh, và vượt kernel attention tiết kiệm bộ nhớ 1,88 lần ở độ dài 8192 cho lượt tiến và lùi. Ablation cho thấy cửa sổ suy giảm và bậc đệ quy là hai thành phần có ảnh hưởng đo được, còn kích hoạt sine và positional embedding thì không phân biệt được ở số seed hiện có.

Đóng góp đề xuất cho kết quả hai mặt. Khoảng giá trị của tham số suy giảm thực sự quan trọng: chuyển từ một khoảng đặt tùy tiện sang một lựa chọn có nguyên tắc thu được khoảng 1 PPL. Nhưng việc suy khoảng đó từ thống kê corpus không mang lại lợi ích đo được so với một lựa chọn cách đều theo thang log, và phần chênh lệch nhỏ còn lại xuất hiện đồng đều trên cả hai ngôn ngữ. Chúng tôi báo cáo kết quả phủ định này cùng toàn bộ giới hạn của bản tái hiện, vì chính nhánh đối chứng do chúng tôi thiết kế đã dự đoán và sau đó xác nhận nó.

References

- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Daniel Y. Fu, Tri Dao, Khaled K. Saab, Armin W. Thomas, Atri Rudra, and Christopher Ré. 2023. Hungry hungry hippos: Towards language modeling with state space models. In *International Conference on Learning Representations (ICLR)*.
- Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.
- Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.
- Albert Gu, Karan Goel, and Christopher Ré. 2022. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*.
- Harsh Mehta, Ankit Gupta, Ashok Cutkosky, and Behnam Neyshabur. 2022. Long range language modeling via gated state spaces. *arXiv preprint arXiv:2206.13947*.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*.
- Dat Quoc Nguyen and Anh Tuan Nguyen. 2020. PhoBERT: Pre-trained language models for Vietnamese. In *Findings of the Association for Computational Linguistics: EMNLP 2020*.
- Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y. Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. 2023. Hyena hierarchy: Towards larger convolutional language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*.
- David W. Romero, Anna Kuzina, Erik J. Bekkers, Jakub M. Tomczak, and Mark Hoogendoorn. 2022. Ckconv: Continuous kernel convolution for sequential data. In *International Conference on Learning Representations (ICLR)*.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. 2020. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.